

MIDI Maze ROM – Complete Serial Communications Analysis

This document extracts and annotates all serial communication–related code from the MIDI Maze cartridge ROM. The routines implement a custom POKEY serial driver that bypasses Atari OS SIO data transfer while still using the OS serial interrupt vectors.

Overview

MIDI Maze installs its own handlers for VSERIN and VSEROR, programs POKEY directly, and uses interrupt-driven ring buffers for RX and TX. The serial core consists of: an init routine, RX/TX ISRs, and blocking send/receive APIs.

Serial Input ISR (VSERIN)

```
.byte $98          ; TYA
LBE9D    PHA
          LDA SERIN
          LDY rx_w
          STA rx_buf,Y
          INC rx_w
          PLA
          TAY
          PLA
          RTI
```

Serial Output ISR (VSEROR)

```
.byte $98          ; TYA
LBEAD    PHA
          LDY tx_r
          CPY tx_w
          BEQ empty
          LDA tx_buf,Y
          STA SEROUT
          INC tx_r
          JMP done
empty:
          LDA #0
          STA tx_active
done:
          PLA
          TAY
          PLA
          RTI
```

Send-byte API

```
SEI
if tx_active == 0:
    write SEROUT
    tx_active = 1
else:
    enqueue byte
CLI
```

Receive-byte API

```
while rx_r == rx_w:
    wait
return rx_buf[rx_r++]
```

Serial Init Routine

- Disable IRQs
- Patch VSERIN/VSEROR
- Clear ring buffers
- AUDCTL = \$28
- ADF3/4 = \$0015
- SKCTL = \$13
- Enable serial IRQs
- Re-enable IRQs

Key Observations

- RX and TX are fully interrupt-driven. - MIDI Maze does not clear SKREST per byte (MIDIMATE does). - TX IRQs are not gated off when idle. - AUDCTL/SKCTL differ from MIDIMATE handler. - ISR stack usage relies on OS serial vector conventions.